you saw in figure 24.2, this metadata is made available to the `AuthorizationMiddleware` when an endpoint is selected by the `RoutingMiddleware`.

You can apply the `[Authorize]` attribute at the action scope, controller scope, Razor Page scope, or globally, as you saw in chapter 21. Any action or Razor Page that has the `[Authorize]` attribute applied in this way can be executed only by an authenticated user. Unauthenticated users will be redirected to the login page.

TIP There are several ways to apply the `[Authorize]` attribute globally. You can read about the options and when to choose which option on my blog: http://mng.bz/opQp.

Sometimes, especially when you apply the `[Authorize]` attribute globally, you might need to poke holes in this authorization requirement. If you apply the `[Authorize]` attribute globally, any unauthenticated requests are redirected to the login page for your app. But if the `[Authorize]` attribute is *global*, when the login page tries to load, you'll be unauthenticated and redirected to the login page again. And now you're stuck in an infinite redirect loop.

To get around this, you can direct specific endpoints to ignore the `[Authorize]` attribute by applying the `[AllowAnonymous]` attribute to an action or Razor Page, as shown in the next listing. This allows unauthenticated users to execute the action, so you can avoid the redirect loop that would otherwise result.

Listing 24.2 Applying `[AllowAnonymous]` to allow unauthenticated access

```
[Authorize]                                          ❶
public class AccountController : ControllerBase
```